RICE UNIVERSITY

Message Passing

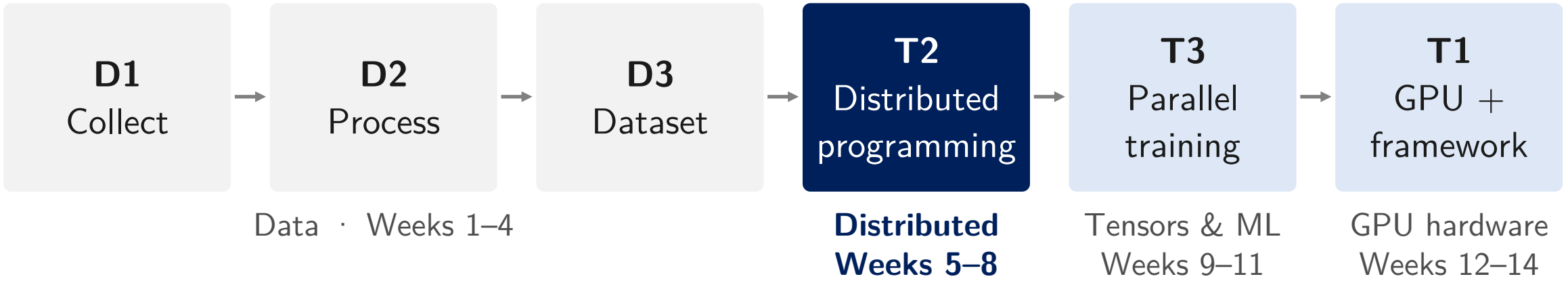
Processes, ranks, messages, and collectives — and a two-number model of what they cost

Yuke Wang

Rice University

COMP 329 · Spring 2027 · Week 7, Lecture 1

Where we are in the pipeline



In our pipeline

Today: many processes that cooperate by **messages**. **D3 is due Friday**; T2 and Lab 4 are released after the next lecture.

Next: collectives (Week 7 L2) → all-reduce rings and trees (Week 8 L1) → NCCL (Week 8 L2)

Why one GPU is not enough: memory

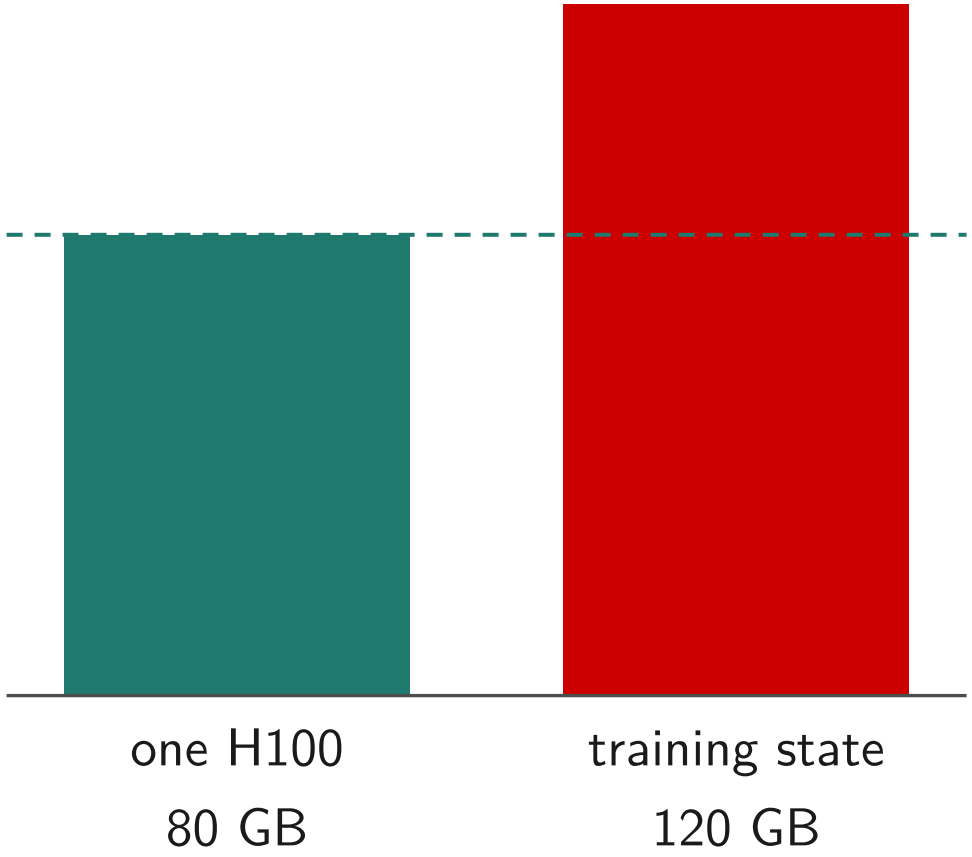
Can one 80 GB GPU train a 7.5-billion-parameter model?

parameter: one number that training learns. Training with Adam keeps $\approx$ **16 bytes** per parameter.

$$7.5 \times 10^9 \times 16 \text{ bytes} = 120 \text{ GB}$$

Watch out

The training state alone exceeds one GPU. **Split it** across many processes.



Source: Rajbhandari et al., ZeRO, SC 2020 (16 bytes per parameter); NVIDIA H100 datasheet (80 GB); 120 GB derived.

Why one GPU is not enough: time

And how long would the same model take to train? (assumptions in the notes)

$$\text{one GPU:} \quad \approx 7.2 \text{ years}$$

$$\text{1,024 GPUs, perfect scaling:} \quad \approx 2.6 \text{ days}$$

In our pipeline

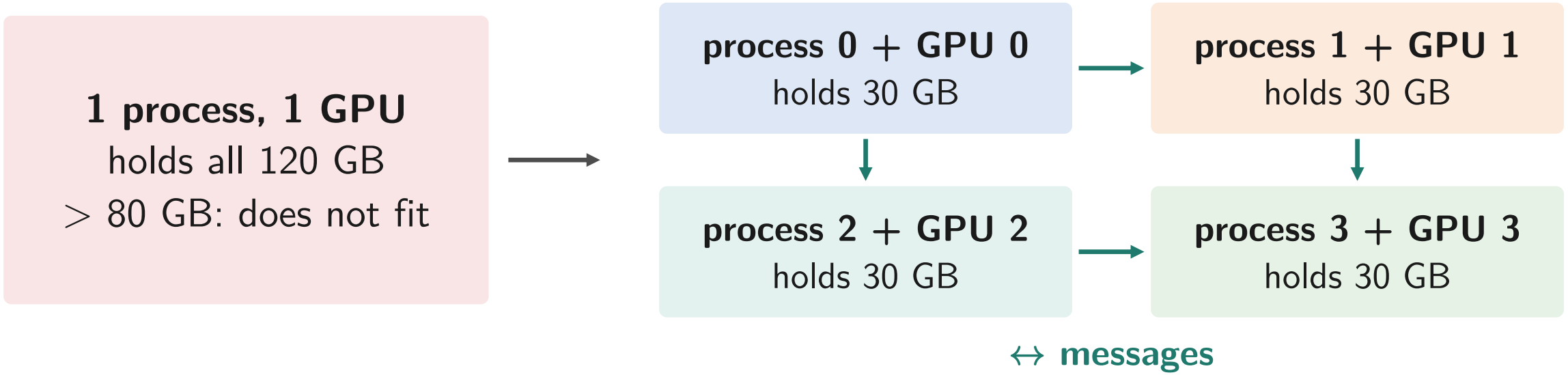
"Perfect scaling" needs the GPUs to **communicate** cheaply every step — today's subject.

Source: Kaplan et al., 2020 ($C \approx 6ND$); H100 datasheet (BF16, dense); 40% utilization assumed; times derived.

What a distributed program does

process: one running copy of a program, with its own memory (and usually its own GPU).

message: data one process sends to another over a link.



Key idea

Split the work and the state across processes; pay for it with **communication**.

Source: Rajbhandari et al., ZeRO, SC 2020; 30 GB per process derived.

Where distributed training shows up

System	Who talks to whom	In this course
MPI (mpi4py)	CPU processes, any machines	T2 all-reduce; Lab 4
NCCL (torch.distributed)	GPUs, in and across servers	T2 benchmark; Week 8
DDP, tensor, pipeline	many GPUs train one model	T3; Weeks 9–10

Llama 3 405B: trained on up to **16,384 H100s**; its training state alone needs the memory of $\approx$ **81 GPUs** (derived).

Why we need it

One GPU is too **small** to hold a large model's state and too **slow** to train it in time. Many processes, each with a GPU, cooperating by **messages**, fix both.

Source: Dubey et al., "The Llama 3 Herd of Models," 2024; ZeRO, SC 2020 (16 B/param); MPI 4.1 standard; NCCL docs.

Learning goals for today

- **Explain** why training needs many processes, and how each finds its **rank** and its **GPU**.
- **Send** and receive messages, and explain how a program can **deadlock**.
- **Draw** what each collective leaves on every rank; build **all-reduce** from two of them.
- **Fit** $T(n) = \alpha + n/\beta$ to measurements and **predict** a message time.

A puzzle to start

Option A
1,000 messages of 1 KB

Option B
1 message of 1 MB

Each message costs 1 µs to start, then moves 10 GB/s. Which option is faster, and by how much?

Answer

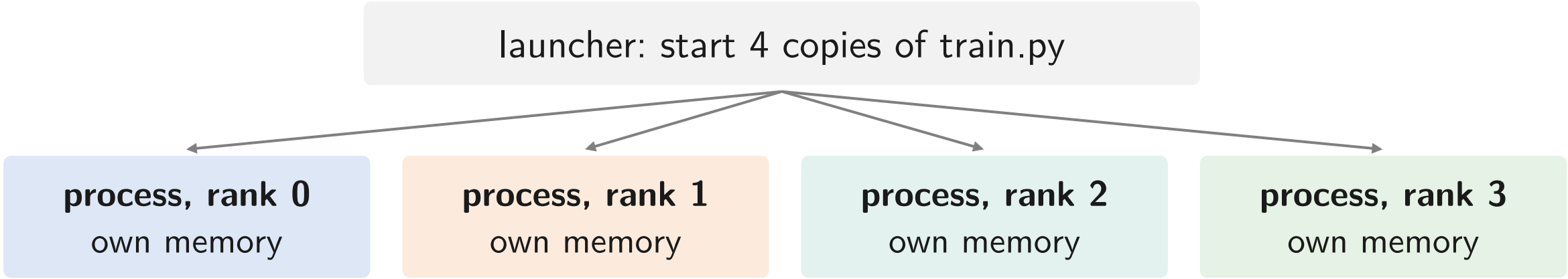
A: **1.1 ms**. B: $\approx$ **0.1 ms**. Same bytes, about **11×** apart: the start-up cost dominates small messages.

Part 1

Processes and ranks

How does each process know its job and its GPU?

One program, many processes




```
rank = comm.Get_rank()      # which one am I?
p = comm.Get_size()         # how many of us?
mine = data[rank::p]        # my share of the data
```

rank: my number, 0 … p − 1.

p (world size): how many.

Key idea

Same code everywhere; behaviour differs only through **rank**. No shared memory: processes cooperate **only by messages**.

Rank and local rank

node 0 (one machine)

GPU 0	GPU 1	GPU 2	GPU 3
rank 0 local 0	**rank 1** local 1	**rank 2** local 2	**rank 3** local 3

node 1 (one machine)

GPU 0	GPU 1	GPU 2	GPU 3
rank 4 local 0	**rank 5** local 1	**rank 6** local 2	**rank 7** local 3

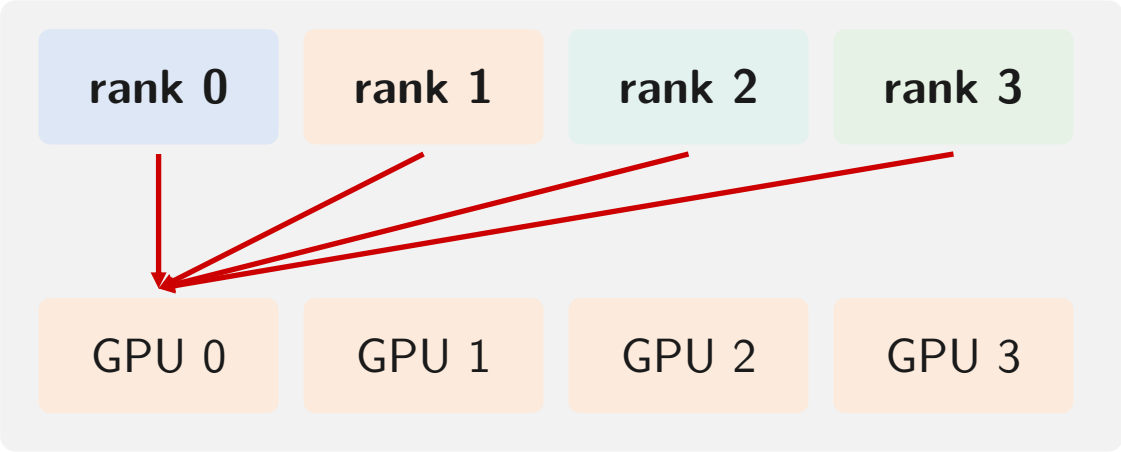
$$\text{rank} = \text{node} \times 4 + \text{local rank}, \qquad p = 2 \times 4 = 8$$

Key idea

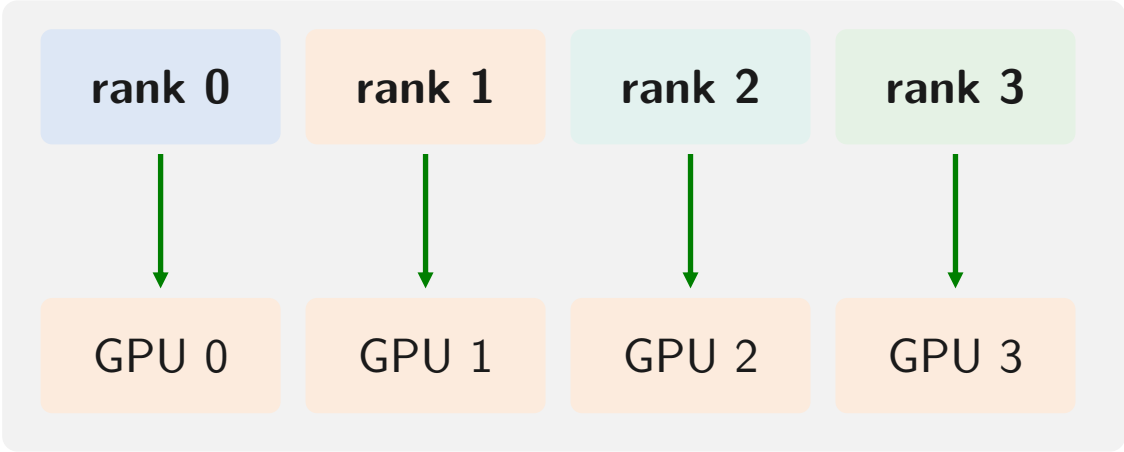
rank picks the work; **local rank** picks the GPU on this machine.

Mapping ranks to GPUs

default: everyone on GPU 0



after calling set_device




```
torch.cuda.set_device(local_rank)    # first!
```

one line, before any GPU work

Watch out

Forget it and four processes share **GPU 0**: out of memory or slow, three GPUs idle.

Source: PyTorch Distributed Overview; torch.cuda.set_device documentation.

Check yourself

Two minutes with a neighbour.

- 4 machines $\times$ 8 GPUs, one process per GPU: world size? Local rank of rank 13?
- A script forgets `set_device(local_rank)` on an 8-GPU machine. What do you observe?
- Every rank sets `x = 0`; then rank 0 sets `x = 5`. What is `x` on rank 1?

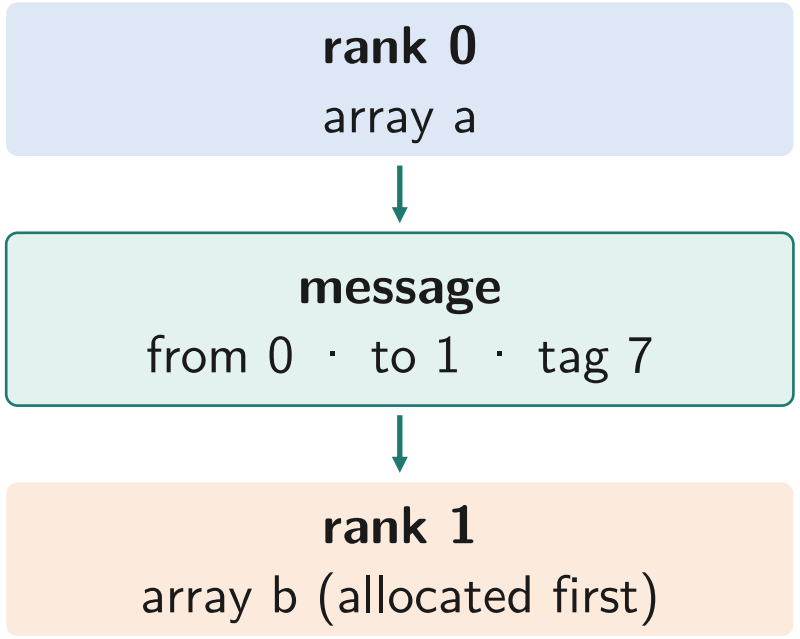
Part 2

Messages and collectives

How do processes exchange data, and what does each group operation compute?

Send and receive

```
rank = comm.Get_rank()
if rank == 0:
    a = np.arange(1000, dtype=np.float64)
    comm.Send(a, dest=1, tag=7)
elif rank == 1:
    b = np.empty(1000, dtype=np.float64)
    comm.Recv(b, source=0, tag=7)
```



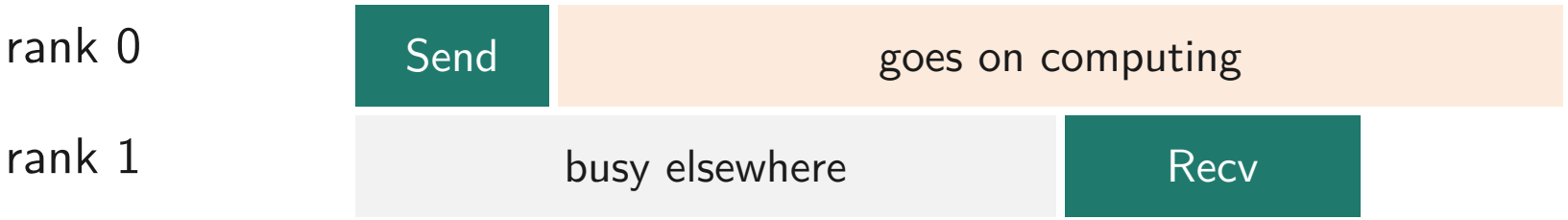
Key idea

tag: a label the sender picks. A receive takes the message whose **source and tag** match; the bytes land in **your** array.

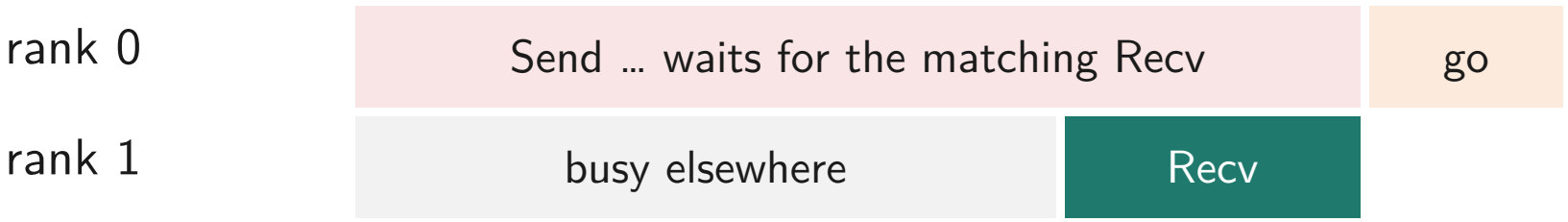
Source: MPI Forum, MPI: A Message-Passing Interface Standard, Version 4.1 (2023); mpi4py documentation.

What “blocking” means

small message



large message



Watch out

Send returns when its array **may be reused**. Large messages wait for the receiver — write code that works **either way**.

Source: MPI Forum, MPI: A Message-Passing Interface Standard, Version 4.1 (2023); mpi4py documentation.

Deadlock

rank 0	rank 1
`Send(a, dest=1)`	`Send(a, dest=0)`
`Recv(b, source=1)`	`Recv(b, source=0)`



each Send waits for a Recv

that comes after the other's Send

Fix

Order by rank: rank 0 sends, then receives; rank 1 receives, then sends.

one laptop run: 1 KB finished · 64 KB **hung**

Watch out

Deadlock: every process waits for another, forever. Small tests can hide it.

Source: one laptop run made for these slides: 2 processes, MPICH 5.0.2 shared memory, mpi4py 4.1.2.

What is a collective?

- A **collective** is called by **every** rank of the group, in the **same order**.
- Some have a **root**: the one rank that provides the data or receives the result.
- A **reduction** combines the ranks' arrays elementwise, e.g. by **sum**.

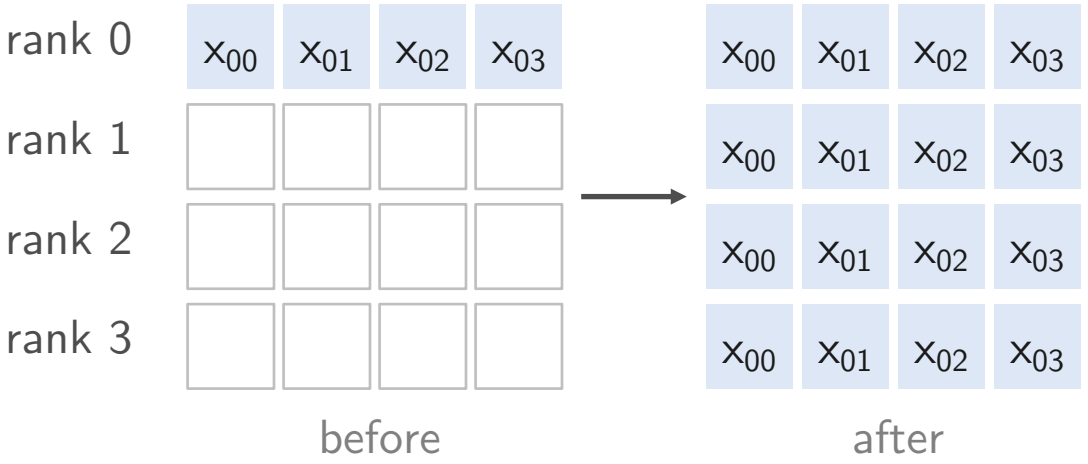
Watch out

If one rank skips or reorders a collective, the others wait **forever**.

Source: MPI standard 4.1, Chapter 6.

Broadcast

Broadcast (root: rank 0)



the root's array is copied to everyone

Use

Start every copy of a model from the **same weights** (Week 9).

Key idea

Broadcast: every rank ends with **all** of the root's data.

Source: MPI Forum, MPI: A Message-Passing Interface Standard, Version 4.1 (2023); mpi4py documentation.

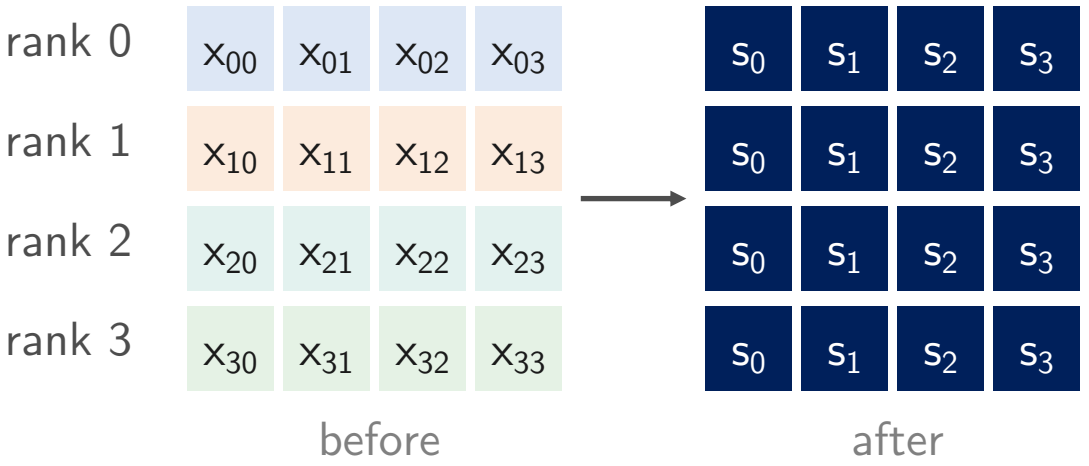
Reduce and all-reduce

Reduce, sum (root: rank 0)



the sum lands on the root only

All-reduce, sum



the sum lands on every rank

Key idea

$s_0 = x_{00} + x_{10} + x_{20} + x_{30}$, elementwise. All-reduce: the sum on **every** rank.

Source: MPI Forum, MPI: A Message-Passing Interface Standard, Version 4.1 (2023); mpi4py documentation.

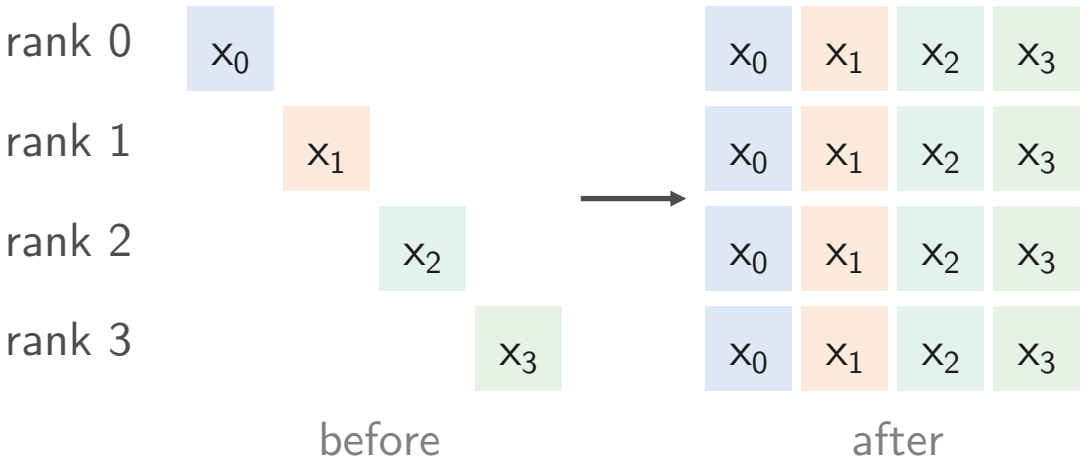
Reduce-scatter and all-gather

Reduce-scatter, sum



rank i keeps chunk i of the sum

All-gather



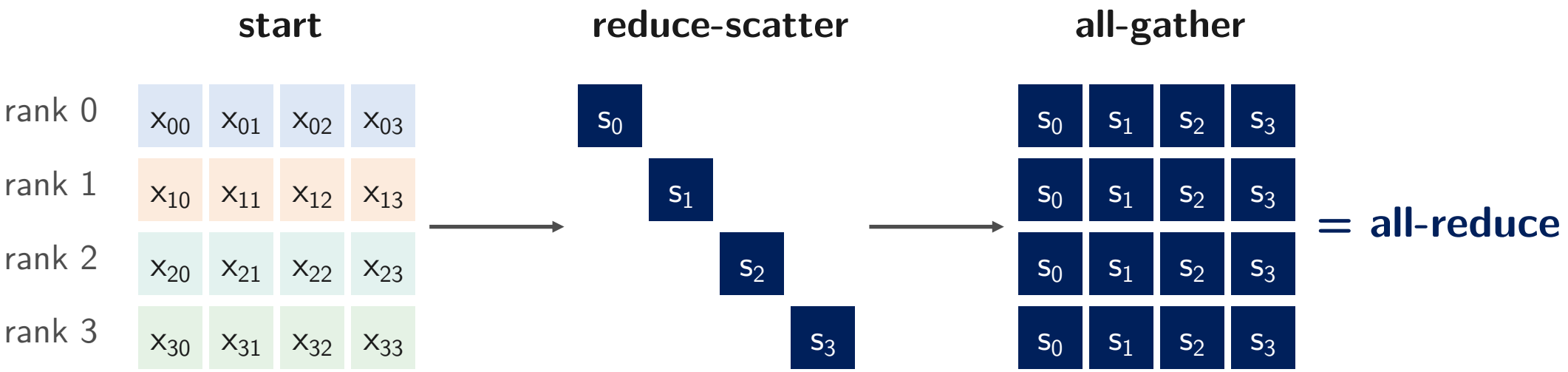
every rank gets every chunk

Key idea

Reduce-scatter: each rank ends with **one chunk** of the sum. All-gather: each rank ends with **all** chunks.

Source: MPI Forum, MPI: A Message-Passing Interface Standard, Version 4.1 (2023); mpi4py documentation.

All-reduce = reduce-scatter + all-gather

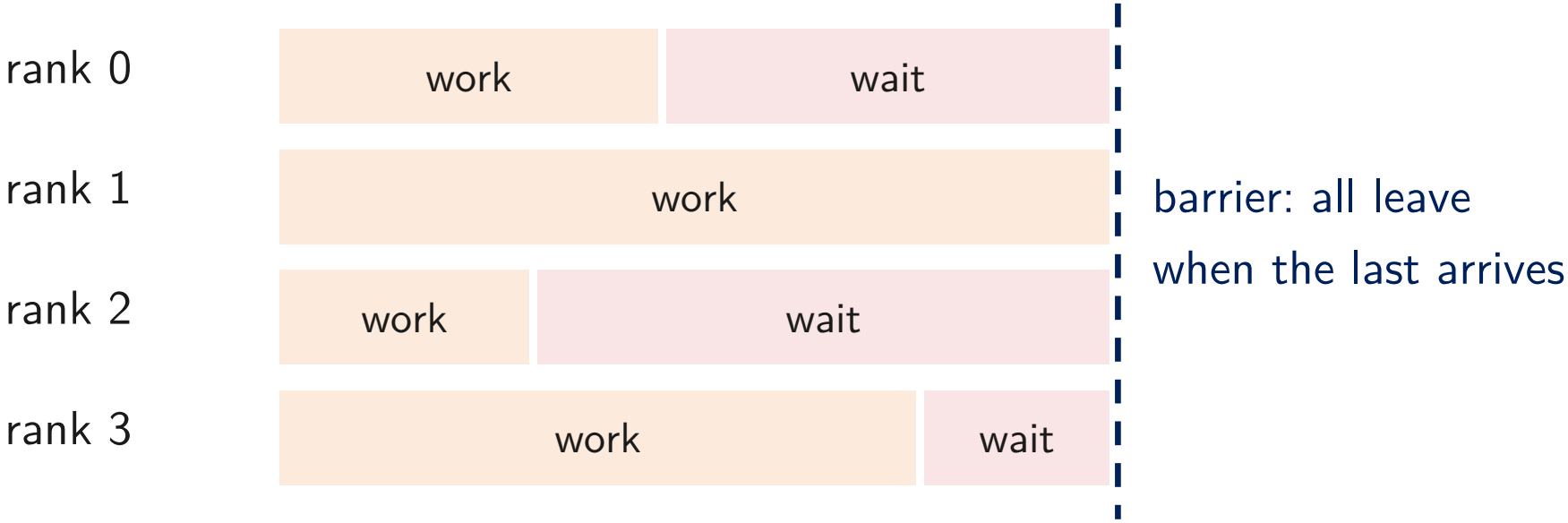


Key idea

Two steps, and **every** rank is busy in each — the idea behind the fast ring algorithm.

Source: Thakur, Rabenseifner & Gropp, "Optimization of Collective Communication Operations in MPICH," IJHPCA 19(1), 2005.

Synchronization: the barrier



Key idea

A **barrier** lines everyone up — use it before starting a timer (as in the BUILD). Other collectives may not wait for everyone.

Source: MPI Forum, MPI: A Message-Passing Interface Standard, Version 4.1 (2023); mpi4py documentation.

A first use: averaging gradients

```
g = local_gradient()        # my share of the batch
total = np.empty_like(g)
comm.Allreduce(g, total, op=MPI.SUM)
g_avg = total / p           # same on every rank
```

average of the ranks' gradients

= gradient of the average loss

In our pipeline

T2 builds this all-reduce from **Send** and **Recv**; T3 trains a model with it.

Check yourself

Two minutes with a neighbour.

- Both ranks call `Send` then `Recv` to each other. When does it work? Is it correct?
- Four ranks, each with a 1,000-element array, call reduce-scatter. What does each get?
- Ranks 0, 1, 3 call all-reduce; rank 2 calls broadcast first. What happens?

Part 3

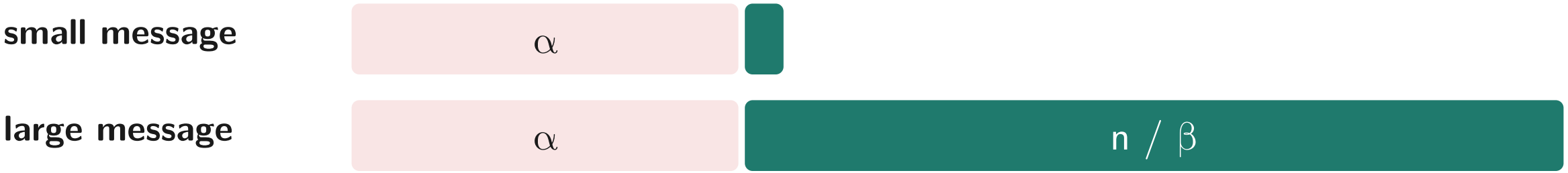
The α–β cost model

Two numbers that predict the time of any message

The α–β model

$$T(n) = \alpha + n \,/\, \beta$$

- α (latency): the fixed start-up cost of **every** message, in seconds.
- β (bandwidth): bytes per second once the data flow.



Key idea

Small messages are **latency-bound** (pay α); large ones are **bandwidth-bound** (pay n/β).

Source: Hockney, Parallel Computing 20(3), 1994; Thakur, Rabenseifner & Gropp, IJHPCA 2005.

Where small turns into large

$$\alpha = n \,/\, \beta \quad \Leftrightarrow \quad n^* = \alpha \cdot \beta$$

below n*: latency wins · above n*: bandwidth wins · at n*: half of β

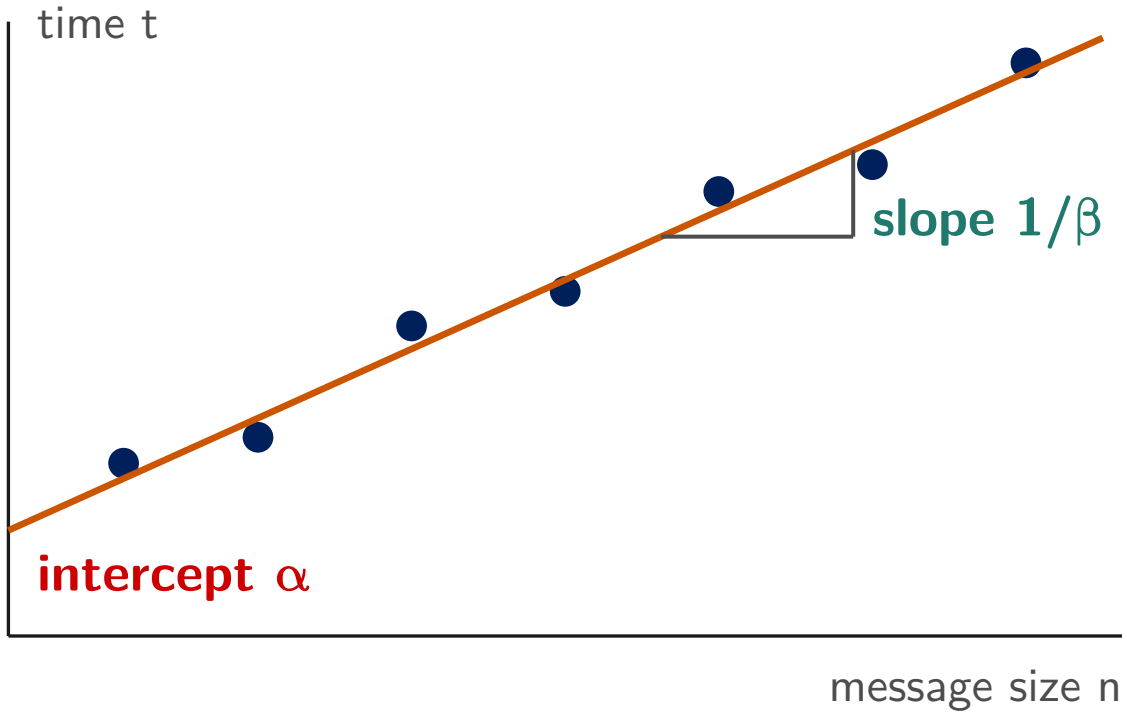
$$\alpha = 1\ \mu s,\ \ \beta = 10\ GB/s \quad \Rightarrow \quad n^* = 10\ KB$$

Key idea

Messages much smaller than $\mathbf{n^* = \alpha\beta}$ waste the link; send **fewer, larger** messages.

Source: Hockney, Parallel Computing 20(3), 1994 (half-performance message size).

Fitting α and β: a straight line

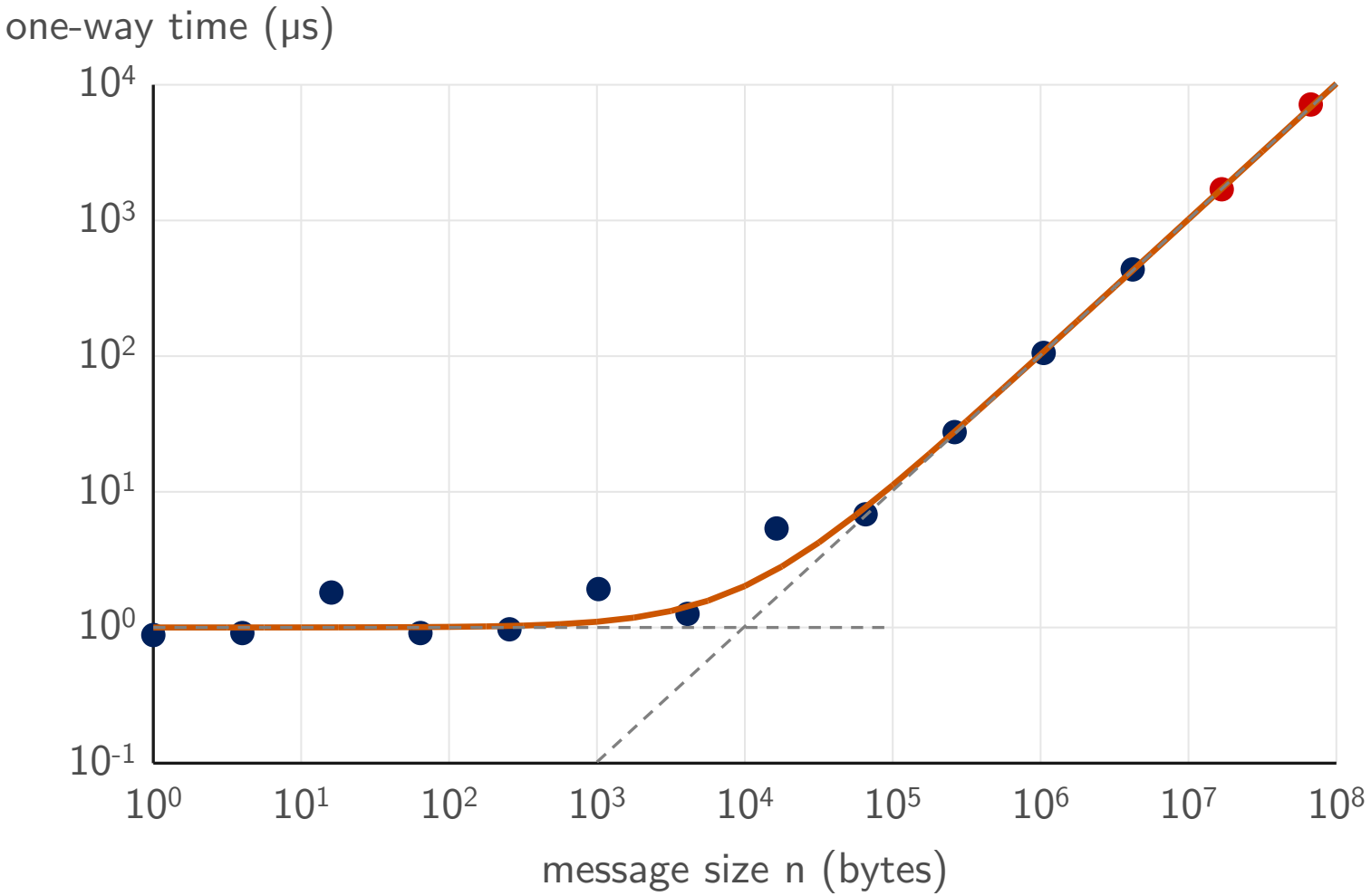


- Measure t at many sizes n.
- Fit the best straight line: **least squares**.
- Read off α and β.

Key idea

Fitting α and β is ordinary **linear regression** of time on size.

One laptop's ping-pong



Fit (blue points)

$\alpha \approx$ **1.0 µs**

$\beta \approx$ **9.8 GB/s**

blue: used in the fit

red: kept out

orange: fitted model

Source: one laptop run made for these slides: 2 processes, MPICH 5.0.2 shared memory, mpi4py 4.1.2.

Predict, then measure

64 MiB $= 2^{26}$ bytes — a size the fit never saw

$$\text{predicted:}\quad \alpha + 2^{26} \,/\, \beta \;\approx\; 6.8\ \text{ms}$$

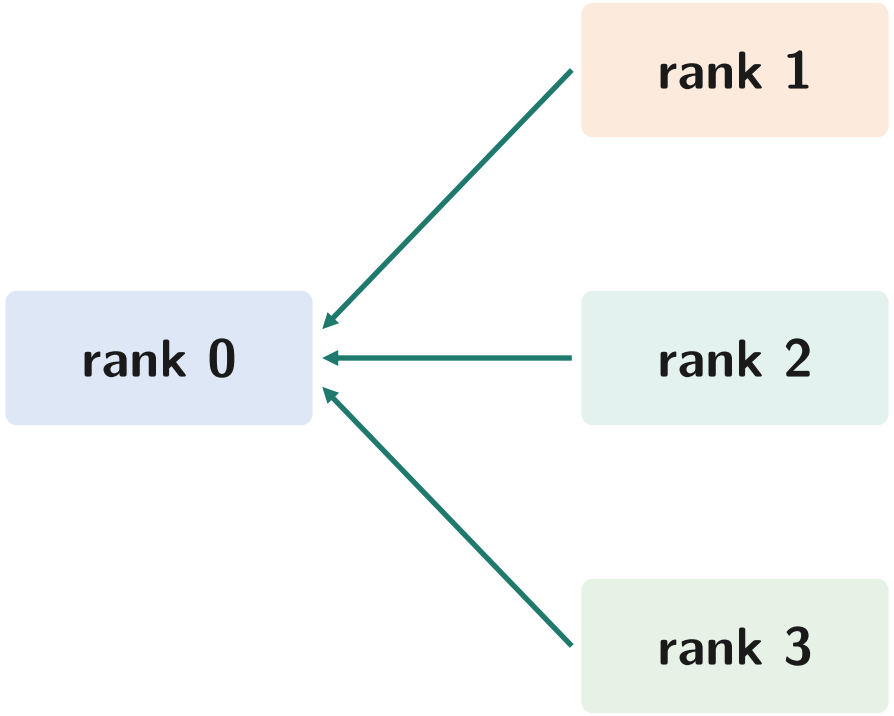
$$\text{measured (one laptop run):}\quad 7.2\ \text{ms}$$

Check

About **4%** off: two numbers predict a message we never measured. Always predict **before** you measure.

Source: one laptop run made for these slides: 2 processes, MPICH 5.0.2 shared memory, mpi4py 4.1.2.

A naive all-reduce



$$T = 2\,(p-1)(\alpha + n\,/\,\beta)$$

Watch out

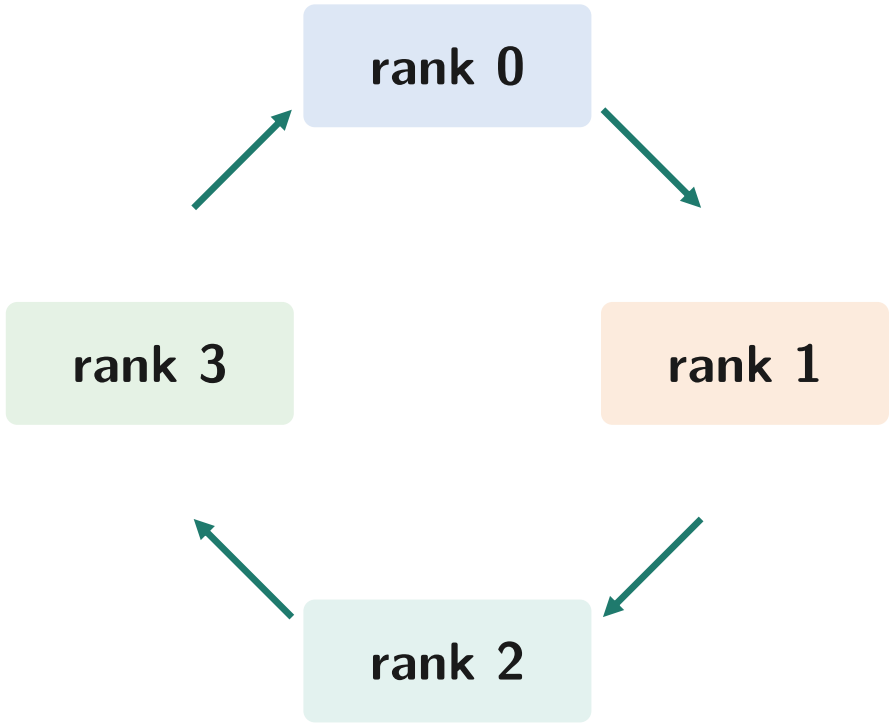
Rank 0's link carries **everything**: time grows **with p**.

p − 1 arrays in, one after another;

then p − 1 sums back out

Source: Thakur, Rabenseifner & Gropp, "Optimization of Collective Communication Operations in MPICH," IJHPCA 19(1), 2005.

Preview: the ring all-reduce



each step: every rank sends one small chunk to its neighbour, all at once

2 GB of gradients, 8 GPUs (NVLink):

All-reduce	Time
Naive (via rank 0)	$\approx$ 62 ms
Ring	$\approx$ 7.8 ms

Check

Same answer, **p× less time**: use every link at once. Details in Week 8.

Source: Thakur, Rabenseifner & Gropp, IJHPCA 2005; NVLink from the H100 datasheet; idealized.

Check yourself

Two minutes with a neighbour.

- $\alpha = 5$ µs, $\beta = 25$ GB/s. What is n*? Is a 1 KB message latency- or bandwidth-bound?
- Same link: predict the time of a 100 MB message.
- From 8 to 16 GPUs: how does each all-reduce's time change for a large array?

IN-CLASS BUILD

Ping-pong and an α–β fit

Pairs: measure, fit, then predict a message you have not measured.

Exercise 1: ping-pong

```
buf = np.zeros(n, dtype=np.uint8)   # n bytes
comm.Barrier(); t0 = MPI.Wtime()
for _ in range(reps):
    if rank == 0: comm.Send(buf, dest=1)
    comm.Recv(buf, source=1 - rank)
    if rank == 1: comm.Send(buf, dest=0)
t = (MPI.Wtime() - t0) / reps / 2   # one-way
```

DO

1. Predict t for 1 byte and for 16 MiB.
2. Run it on 2 processes, 1 byte to 16 MiB.
3. Save n and t to ping.txt.

ASK

Why can this loop never deadlock?

Source: mpi4py documentation; code run on one laptop (MPICH 5.0.2, mpi4py 4.1.2).

Exercise 2: fit, then predict

```
n, t = np.loadtxt("ping.txt", unpack=True)
A = np.column_stack([np.ones_like(n), n])
w = 1 / t                          # weight sizes equally
c, *_ = np.linalg.lstsq(A * w[:, None], t * w,
                        rcond=None)
alpha, beta = c[0], 1 / c[1]
print(alpha + 2**26 / beta)        # predict 64 MiB
```

DO

Write the 64 MiB prediction **before** measuring it. Then measure: how far off?

STRETCH

Run the ping-pong twice. Which changes more between runs, α or β?

Source: NumPy documentation, numpy.linalg.lstsq; code run on the laptop data of Part 3.

Solutions 1: the ping-pong

- **No deadlock**: rank 1 always receives **before** it sends.
- Shape: flat for small n ($\approx \alpha$), then growing in proportion to n ($\approx n/\beta$).
- One laptop run: 1 byte $\approx$ **0.9 µs**, 16 MiB $\approx$ **1.7 ms**.

In our pipeline

Your numbers will differ, especially the small ones; the **shape** will not.

Source: one laptop run made for these slides: 2 processes, MPICH 5.0.2 shared memory, mpi4py 4.1.2.

Solutions 2: the fit

$$\alpha \approx 1.0\ \mu\text{s}, \quad \beta \approx 9.8\ \text{GB/s}$$

64 MiB: predicted 6.8 ms, measured 7.2 ms

In our pipeline

T2 repeats this fit for **NCCL on GPUs** and for **two servers**.

Source: one laptop run made for these slides: 2 processes, MPICH 5.0.2 shared memory, mpi4py 4.1.2.

Key takeaways

- Large models need **many processes**: the training state alone exceeds one GPU.
- One program, p processes: **rank** picks the work, **local rank** picks the GPU.
- Blocking Send/Recv in the wrong order can **deadlock** — small tests may hide it.
- Collectives: every rank calls them; **all-reduce = reduce-scatter + all-gather**.
- $\mathbf{T(n) = \alpha + n/\beta}$: fit it by least squares, then **predict** before measuring.

Next time, and readings

When	What
Week 7, Lecture 2	Collective communication; T2 and Lab 4 released after it
Friday (this week)	D3 due
Week 8, Lecture 1	All-reduce: rings and trees — the ring in detail

READINGS

Thakur, Rabenseifner & Gropp, "Optimization of Collective Communication Operations in MPICH," IJHPCA 2005.

PyTorch documentation: "PyTorch Distributed Overview".